Hockey Pool Analytics Pipeline

Architecture, Models, Diagnostics, and Results

Julian di Giovanni

2026-07-24

Contents

1 Introduction

1.1 Project purpose

This project produces draft-day player rankings for a custom fantasy hockey pool held once per year. The pipeline is rebuilt annually from a shared codebase with minimal configuration changes, following the principle: *copy last season's config, update a few fields, run* `--stage all`.

The output is an Excel workbook and set of CSVs ranking every NHL player by predicted pool points for the upcoming season, broken down by position (forwards, defense, goalies) and combined into an overall ranking.

1.2 Pool scoring rules

Scoring rules are encoded in `<season>/config.yaml` under the `scoring:` key and are the same year-to-year. Table 1 summarises the current rules.

Table 1: Pool scoring rules (2026-27 season)

Position	Event	Points
Forwards	Goal	1
	Assist	1
Defense	Goal	1
	Assist	1
	Plus/minus	1 per unit
Goalies	Win (non-shutout)	1
	Shutout win (total, not additive)	3
	Best GAA among $\geq$40 GP qualifiers	+10
	Best save% among $\geq$40 GP qualifiers	+10

1.3 Annual workflow

1. **July** (off-season): run `--stage all` to refresh historical data through the just-completed season and generate 2026-27 predictions.
2. **Draft day**: use the output rankings and XLSX workbook to make pick decisions.
3. **Following July**: copy `config.yaml` to the new season folder, update `season`, `season_id`, `roster_as_of_date`, and any `trade_overrides`. Run `--stage all`.

2 System Architecture

2.1 Folder layout

```
common/                shared library, reused every season
  config.py              SeasonConfig dataclass (loads config.yaml)
  scrape/
    nhl_api.py             NHL API client (stats, rosters, game logs)
    rosters.py             current-team lookup (handles off-season moves)
    sources/
      moneypuck.py           MoneyPuck CSV scraper (xG, shot quality,
          GSAx)
  clean/
    merge.py               merges skater/goalie/team data into model CSVs
```

```
  features/
    engineering.py        lag features, EB shrinkage, GSAx, leakage
       guard
  models/
    training.py           5-algorithm zoo, CV, feature selection, OOS
       eval
    forwards.py           forward target + feature config
    defense.py            defense target + feature config
    goalies.py            goalie target + feature config
    pool_ranking.py       scoring, blending, ranking, output writing
  diagnostics/
    reconcile.py          cross-source reconciliation (NHL API vs
       MoneyPuck)
    model_report.py       metrics table, PvA plots, rolling OOS plot
  pipeline.py             stage orchestration (scrape/clean/train/
     predict)

run_season.py         CLI entry point

<season>/             one directory per draft year, e.g. 2026-27/
  config.yaml           season parameters and scoring rules
  data/{raw,processed}/ scraped + cleaned data (gitignored)
  models/               persisted joblib artifacts (gitignored)
  results/              rankings, XLSX, report, diagnostics CSV (
     gitignored)
  plots/                PvA and rolling-OOS plots (gitignored)
```

Listing 1: Repository structure

2.2 Pipeline stages

The four stages run in sequence via:

```
python3 run_season.py --season 2026-27 --stage all
# or individually:
python3 run_season.py --season 2026-27 --stage scrape
python3 run_season.py --season 2026-27 --stage clean
python3 run_season.py --season 2026-27 --stage train
python3 run_season.py --season 2026-27 --stage predict
```

`scrape` Pull skater/goalie stats, team stats, rosters, and game logs from the NHL API; pull xG/shot-quality/GSAx data from MoneyPuck. Seed from prior season's raw directory to avoid re-downloading unchanged historical seasons.

`clean` Merge position stats with team context; compute per-game rates; output `skater_team_data.csv` and `goalie_team_data.csv`.

`train` Run feature engineering, train the 5-algorithm zoo for each target, select the best model, persist to `models/<target>.joblib`.

`predict` Load persisted models, append prediction rows for the target season, score pool points, write rankings, generate diagnostics. Runs in $\sim$2s (no retraining).

3 Data Sources

3.1 NHL API (primary)

Endpoint: `api.nhle.com` and `api-web.nhle.com`. Free, no authentication key required. Historical data from the 2008-09 season onward.

Data pulled per season: skater counting stats (goals, assists, shots, TOI, hits, blocks, PIM, plus/minus), goalie stats (wins, losses, OTL, shutouts, saves, shots against, GAA, save%), team-level stats (goals for/against, points, games played), and current-season per-game logs.

Roster snapshots are pulled as of `roster_as_of_date` in the season config, so off-season trades, free-agent signings, and waiver moves are reflected automatically.

3.2 MoneyPuck (secondary)

Source: `moneypuck.com/data.htm` — free CSV downloads, no key required. All 18 seasons (2008–2025) successfully fetched: 81,355 skater rows, 8,510 goalie rows.

MoneyPuck provides metrics unavailable from the NHL API:

- **Expected goals (xG)**: shot-quality-adjusted goal probability per shot
- **Danger-zone shot counts**: high / medium / low danger breakdowns
- **gameScore**: composite single-number performance index
- **GSAx** (goalie-specific): goals saved above expected

Join strategy: The integer `player_id` is identical in both sources. Join key: `player_id` (integer) + season start year. No name-matching is used. Match rate: ∼94.5%. The remaining ∼5.5% are fringe players genuinely absent from MoneyPuck's records (very few games played). A name-fallback join is attempted for ID misses only.

Rate-limit handling: 1 s between year fetches, 3 s between player entities, 15 s retry on HTTP 429.

3.3 Cross-source reconciliation

A dedicated diagnostic script (`common/diagnostics/reconcile.py`) validates agreement between NHL API counting stats and MoneyPuck. Results for the full historical dataset: Pearson $r \geq 0.9999$ for goals, assists, points, and games_played. The few games_played outliers in earlier seasons trace to historical surname collisions in MoneyPuck (e.g. two players named "Jones" in the same season) — the player_id join prevents these from affecting the model.

3.4 Sources not used

Natural Stat Trick
: Skipped. Free but requires a manually-approved access key before automated pulls. Its stats mostly overlap MoneyPuck's.

Evolving-Hockey
: Skipped. The valuable parts (RAPM, GAR, projections, CSV downloads) are paywalled; the free tier adds nothing beyond MoneyPuck.

Elite Prospects
: Skipped. Bio/prospect data; not stats-focused; scraping ToS is unclear.

4 Feature Engineering

All feature engineering lives in `common/features/engineering.py`.

4.1 Lag features

The core predictor of next-season performance is last-season (and two-seasons-ago) performance. All individual statistics, MoneyPuck metrics, and team-context columns are lagged before use:

$$x_{i,t}^{\text{lag}k} = x_{i,t-k}, \quad k \in \{1, 2\} \tag{1}$$

Lags are computed via a vectorized `groupby("player_id").shift(k)`, producing columns `<feature>_lag1` and `<feature>_lag2` for every eligible feature. Rows missing all core lag columns (players with no prior history) are dropped from training.

4.2 Leakage prevention

The feature selector uses an **allowlist**, not a denylist. A column is eligible as a model predictor only if it satisfies at least one of:

1. Contains `"lag"` in its name (i.e. it is a genuine lag column), or
2. Contains `"covid"` in its name (COVID season indicators), or
3. Contains any of the `_ENGINEERED_MARKERS` substrings: `_career_avg`, `_hist_avg`, `_hist_std`, `_trend`, `_per_game_lag`, `rel_team`, `teammate`, `normalized_team`, `_performance`, `_x_`

Contemporaneous current-season columns never pass this filter (unless explicitly listed and engineered). This allowlist design catches leakage by construction — a new column added to the data schema cannot slip through unless the author intentionally adds it to the markers list.

4.3 Career averages

For high-variance rate statistics, the single most-recent season lag is a noisy estimate. An expanding career average provides a more stable signal:

$$\bar{x}_{i,t}^{\text{career}} = \frac{1}{t-1} \sum_{s<t} x_{i,s} \tag{2}$$

Implemented via `groupby("player_id").shift(1).expanding().mean()` — the shift(1) ensures only seasons prior to t are included (no leakage). Produces columns named `<feature>_career_avg`.

Career averages are computed for:

- **Goalies**: save_pct, GAA, shutouts/game, wins/game, eb_save_pct, GSAx
- **Defense**: plus_minus, plus_minus_per_game

4.4 Empirical Bayes save percentage

Raw season save percentage is a binomial proportion subject to substantial sample-size noise. A backup goalie who faced 300 shots and saved 275 (Sv% = 0.917) has a much wider posterior uncertainty than a starter who faced 1,800 shots at the same rate. The Empirical Bayes (EB) approach (Thomas 2006; community standard in hockey analytics) shrinks each estimate toward the league mean in proportion to the goalie's shot volume.

Model: Assume saves $\sim$ Binomial(n, p) where n = shots faced and p is the true save rate. Place a Beta prior $p \sim \text{Beta}(\alpha, \beta)$. The posterior mean is:

$$\hat{p}_i^{\text{EB}} = \frac{\alpha + \text{saves}_i}{\alpha + \beta + n_i} \tag{3}$$

Prior estimation: α and β are estimated from the data via method of moments on all goalie-seasons with $n_i \geq 100$ shots:

$$\hat{\mu} = \text{mean}(\hat{p}_i), \quad \hat{\sigma}^2 = \text{var}(\hat{p}_i) \tag{4}$$

$$\hat{\alpha} = \hat{\mu}\left(\frac{\hat{\mu}(1-\hat{\mu})}{\hat{\sigma}^2} - 1\right), \quad \hat{\beta} = \hat{\alpha}\,\frac{1-\hat{\mu}}{\hat{\mu}} \tag{5}$$

If the pooled dataset contains fewer than 50 qualifying seasons, the fallback prior $(\alpha, \beta) = (85.0,\ 8.5)$ is used, placing the prior mean at ≈ 0.909 with moderate concentration. The resulting column `eb_save_pct` is then lagged and career-averaged before use.

4.5 GSAx and shot-quality features

Raw GAA conflates shot volume (a team-defense quality) with goalie efficiency. MoneyPuck provides expected goals against based on shot location and type, enabling a cleaner individual-skill metric:

$$\text{GSAx}_i = \text{xGoals\_against}_i - \text{goals\_against}_i \tag{6}$$

A positive GSAx means the goalie *outperformed* the shot difficulty they faced. GSAx is more persistent season-to-season than raw save percentage (published YoY autocorrelation: $r \approx 0.15$–0.30 for GSAx vs $r \approx 0.05$–0.15 for raw Sv%).

Additionally, `hd_shot_pct` = high-danger shots / total shots against captures the shot-mix the defense allowed — a partial control for team quality independent of the goalie.

Both features are lagged and added to career averages.

4.6 COVID and experience features

COVID indicators: The 2019-20 season was suspended mid-year; the 2020-21 season was shortened to 56 games. A binary `covid_year` flag and interaction terms (`<metric>_x_covid`) are added to allow models to down-weight these atypical seasons.

Experience: A polynomial in years played is added for each position:

$$\text{years\_played}, \quad \text{years\_played}^2, \quad \text{years\_played}^3$$

with a position-specific "prime" window (forwards: years 3–10; defense: years 3–12; goalies: years 3–12) encoded as an additional binary indicator.

4.7 Interaction features

`engineer_features()` generates pairwise lag×lag interaction terms between a small set of key metrics (e.g. plus_minus_per_game × points_per_game for defense), and lag × COVID interactions for the most volatile stats. These are allowlisted via the `_x_` marker.

5 Modeling Framework

5.1 Algorithm zoo

Five candidate algorithms are trained for each target. Table 2 summarises the hyperparameter search for each.

Table 2: 5-algorithm zoo and search configuration

Algorithm	Search	Hyperparameter grid
Random Forest	Randomized (8)	`n_estimators` $\in \{150, 250\}$, `max_depth` $\in \{12, 18\}$, `min_samples_split` $\in \{15, 25\}$, `min_samples_leaf` $\in \{8, 12\}$, `max_features` $\in \{\sqrt{p}, 0.4\}$
Gradient Boosting	Randomized (10)	`n_estimators` $\in \{150, 250\}$, `max_depth` $\in \{3, 5\}$, `learning_rate` $\in \{0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$, early stopping: `validation_fraction=0.2`, `n_iter_no_change=10`
Ridge	Grid	`alpha` $\in \{10, 50, 100, 500\}$
Lasso	Grid	`alpha` $\in \{0.1, 0.5, 1.0, 5.0\}$, `max_iter=3000`
ElasticNet	Grid	`alpha` $\in \{0.1, 1.0, 5.0\}$, `l1_ratio` $\in \{0.3, 0.5, 0.7\}$, `max_iter=3000`

Linear models (Ridge, Lasso, ElasticNet) receive a `RobustScaler` prior to fitting. Tree models are scale-invariant and receive no scaling.

5.2 Cross-validation strategy

When the training data has a temporal ordering (season year), a `TimeSeriesSplit` is used, holding out the most-recent block of the training set:

$$n_{\text{folds}} = \min(5,\ \max(3,\ \lfloor n/30 \rfloor)) \tag{7}$$

$$s_{\text{test}} = \min(0.25,\ \max(0.15,\ 40/n)) \tag{8}$$

This ensures a minimum of 40 rows in the held-out test set while never exceeding 25% of the training data. Without temporal ordering, a shuffled `KFold` is used.

5.3 Feature selection

When the feature matrix has more than 20 columns, `SelectKBest` with F-statistic univariate regression is applied before fitting:

$$k = \min(50,\ \max(15,\ \lfloor p/3 \rfloor)) \tag{9}$$

where p is the total number of eligible features. This reduces noise features while retaining at least 15 predictors.

5.4 Model selection score

After fitting each algorithm, a **selection score** balances test-set performance with overfitting:

$$\text{score} = R^2_{\text{test}} - 0.1 \times \frac{R^2_{\text{train}} - R^2_{\text{test}}}{R^2_{\text{train}}} \tag{10}$$

A model with a high train R^2 but poor test R^2 is penalised. The algorithm with the highest selection score is chosen as the production model for each target.

5.5 Model persistence

The winning model for each target is saved to `models/<target>.joblib` via `joblib.dump`. The `--stage predict` step loads these artifacts (∼2s) rather than retraining (∼60s). The `TrainedModel` dataclass stores: `target`, `model_type`, `estimator`, `selected_features`, `scaler`, `metrics`, `best_params`, `y_test`, `y_pred_test` (the latter two enabling diagnostic plots without retraining).

6 Out-of-Sample Validation

6.1 Primary OOS method: train-all vs. exclude-latest

Every target is validated via `train_all_vs_exclude_latest()`:

1. Train **model A** on all available seasons.
2. Train **model B** on all seasons *except* the most recent.
3. Evaluate model B on the held-out most-recent season — this is the out-of-sample (OOS) evaluation.
4. Choose the model with the higher selection score (Eq. 10) for production prediction. In practice, model A (more data) wins most of the time.

6.2 OOS metrics

For each model, the following metrics are reported for both the in-sample holdout and the OOS evaluation:

R^2	Coefficient of determination. Fraction of variance explained. $R^2 < 0$ means the model is worse than predicting the training-set mean.
MAE	Mean absolute error. Scale-dependent; reported in same units as the target.
Baseline R^2	R^2 of a constant predictor (training-set mean). Any model with $R^2 < R^2_{\text{baseline}}$ is flagged `beats_baseline=False`.
Spearman ρ	Rank correlation between predicted and actual values. Range $[-1, 1]$; 0 = no ranking information. Directly measures the pool draft objective.
Concordance	Pairwise ranking accuracy: $C = \Pr[\hat{y}_i > \hat{y}_j \mid y_i > y_j]$. Computed as $(\tau + 1)/2$ where τ is Kendall's τ. Range $[0, 1]$; $C = 0.5$ = chance.
Directional accuracy	Fraction of players where the model correctly predicts whether they are above or below the mean: $\text{dir\_acc} = \Pr[\text{sign}(\hat{y} - \bar{\hat{y}}) = \text{sign}(y - \bar{y})]$.
Bias	Systematic over- or under-prediction: $\bar{\hat{y}} - \bar{y}$.

`top1_match_max` Binary: does the model's argmax (predicted best player) match the actual best player in the OOS year? Analogously `top1_match_min` for the argmin (e.g. the goalie who would win the best-GAA bonus).

Rationale for rank metrics: $R^2 \approx 0$ does not mean the model has zero utility for the pool. The draft is a ranking problem — what matters is whether the model correctly orders players relative to each other, not the absolute predicted value. A model with $R^2 = 0.05$ and concordance $= 0.65$ is genuinely useful.

6.3 Rolling OOS evaluation

An optional gold-standard time-series OOS is implemented in `rolling_oos_eval()`. For each holdout year t (requiring at least 5 prior training years), the model is trained on all years $< t$ and evaluated on year t. This produces an R^2 curve over holdout years, visualised in `plots/diagnostics/rolling_oos_all.png`. It is computationally expensive (one full train per holdout year) and is triggered with the `--rolling-eval` flag.

7 Position-Specific Models

7.1 Forwards

Target: `points_per_game` = (goals + assists) / games_played.

Algorithms: All five (no restriction).

Feature set: All skater individual stats lagged 1 and 2 years; MoneyPuck xG and shot metrics lagged; team context (goals for/against, power-play efficiency) lagged; experience polynomial; COVID interactions. Forwards are split from defensemen before modelling.

Observations: All skaters with at least one full year of lag history in the training data (∼9,000–12,000 rows across 2010–2025).

OOS performance (2026-07-24): $R^2 = 0.665$, concordance ≈ 0.83 — the healthiest target in the pipeline. Points are moderately persistent: elite forwards remain elite.

7.2 Defense

Targets:

- `points_per_game` (goal + assists per game) — same construct as forwards.
- `plus_minus_per_game` — net goal differential while on ice per game.

Algorithms: All five for both targets.

Leakage: `allow_contemporaneous_team=False` for both targets. Contemporaneous team stats (e.g. this season's team goals-for) are NaN at prediction time. Lagged team proxies (`team_goals_for_per_game_lag1`, teammate-quality composites) are used instead.

Additional features: Career averages for plus_minus and plus_minus_per_game, and four lagged teammate-quality proxies (offensive strength, defensive strength, consistency, elite-team indicator).

Prediction blending: Both defense targets are blended post-prediction with 2-year historical per-game averages to correct for model compression at the extremes of the distribution (Section 8).

OOS performance: points/game $R^2 \approx 0.4$, concordance ≈ 0.72; plus_minus/game $R^2 = 0.031$, concordance ≈ 0.54.

7.3 Goalies

Targets:

Target	Internal name	Restriction
Wins per game	`wins_per_game`	All goalies
Shutouts per game	`shutouts_per_game`	All goalies
Goals against avg	`goals_against_average`	≥40 GP qualifiers only
Save percentage	`save_percentage`	≥40 GP qualifiers only

Algorithm restriction for GAA and save%: Only Ridge, Lasso, and ElasticNet (`_LINEAR_ONLY`). Season-to-season autocorrelation for both stats is $r \approx 0.09$ (near zero). Tree models fit this noise and produce predictions that cluster around the training mean; regularized linear models correctly regress toward the mean by design.

Training filter: All goalies (not just qualified ones) are included in training. Previously, only goalies with ≥ 40 GP were used, collapsing the training set from ∼1,100 to ∼340 rows. The QUALIFIED_ONLY distinction now applies only at prediction time (bonus eligibility), not at training time.

Key engineered features for goalies:

- `eb_save_pct`: Empirical Bayes posterior mean (Section 4.4) — more stable than raw save_pct, especially for low-volume goalies.
- `gsax`: Goals saved above expected (Section 4.5) — lagged GSAx more persistent than raw Sv%.
- `hd_shot_pct`: High-danger shot fraction — partial control for team defense quality.
- Career averages for all rate stats including `eb_save_pct` and `gsax`.

OOS performance: wins/game $R^2 \approx 0.10$; shutouts/game $R^2 \approx 0.05$; GAA $R^2 = -0.203$; save% $R^2 = -0.839$. The negative R^2 for precision stats is expected given near-zero autocorrelation — this is a fundamental data property, not a model failure (Thomas 2006). The bonus only needs correct argmax/argmin among qualified goalies, so concordance > 0.5 is the relevant threshold.

8 Prediction Blending for Defense

8.1 Problem: model compression

Gradient boosting (and to a lesser extent other tree models) tends to compress predictions toward the center of the training distribution. As a concrete example, Cale Makar's points_per_game_lag1 = 1.053 was predicted as 0.762 — the model "averaged him toward the pack." Separately, ElasticNet predicted near-constant plus_minus for almost all defensemen (heavy ℓ_1 regularization zeroed most coefficients, leaving predictions clustered at ∼0).

The result was that elite D-men ranked far lower overall than hockey knowledge would justify: Makar at #44 overall, for example.

8.2 Solution: weighted blend with historical average

For each defense target, the model prediction is blended with a 2-year historical per-game average that is already present in the engineered feature set:

$$\hat{y}_i^{\text{blend}} = \alpha \cdot \hat{y}_i^{\text{model}} + (1 - \alpha) \cdot \bar{y}_i^{\text{hist}} \tag{11}$$

If no historical average is available for player i (e.g. a new player in the league), the blend falls back to the raw model prediction.

8.3 Calibrating alpha from OOS concordance

The blend weight α is calibrated from out-of-sample concordance. When concordance = 0.5 (chance), the model adds no ranking information and should receive zero weight. When concordance = 1.0 (perfect ranking), the model should receive full weight. The natural calibration is:

$$\alpha = 2 \times (C_{\text{OOS}} - 0.5) \tag{12}$$

Applied values for the 2026-27 season:

Target	OOS Concordance	α	Interpretation
`defense_points`	≈ 0.72	0.45	Model trusted; hist corrects compression
`defense_plus_minus`	≈ 0.54	0.10	Model barely beats chance; hist dominates

Effect: Makar moved from #44 overall → #4 overall after blending.

9 Pool Scoring and Ranking

9.1 Projected games

Each player's projected games played is derived from a 3-year rolling average of their actual `games_played_player`, capped at `cfg.games_per_season`.

Partial-season filter: Historical seasons with GP < 25 are excluded from the rolling window before computing the average. This prevents late-season rookie call-ups (e.g. a player who plays 15 games in their debut year after being recalled from the AHL in March) from biasing the projection downward. The most-recent season is *always* included regardless of GP — a genuine recent injury should still reduce the projection.

$$\mathcal{H}_i = \{\text{GP}_{i,s} : s < t_{\max},\ \text{GP}_{i,s} \geq 25\} \quad \text{(filtered historical seasons)} \tag{13}$$

$$\text{GP}_i^{\text{proj}} = \min\left(\frac{1}{|\mathcal{W}_i|}\sum_{s \in \mathcal{W}_i} \text{GP}_{i,s},\ \texttt{games\_per\_season}\right), \quad \mathcal{W}_i = (\mathcal{H}_i \cup \{\text{GP}_{i,t_{\max}}\}) \text{ tail-3} \tag{14}$$

For players with no qualifying history: forwards and defense default to 70 games; goalies default to 20 games.

GP-projection diagnostic: On every predict run, a diagnostic compares each player's projected GP to their most-recent actual GP. Players where $\text{GP}_i^{\text{proj}} < 0.70 \times \text{GP}_i^{\text{last}}$ are logged as anomalies and written to `results/diagnostics/gp_projection_check.csv` (all players sorted by ratio, with columns: player_id, player_name, position_group, projected_games, last_season_gp, ratio). After the partial-season fix, only 13 players remain flagged — all with genuinely irregular career patterns (multi-season injuries, suspensions, retirements) rather than call-up artifacts.

9.2 Dual scoring outputs

Two pool-point totals are produced for every player:

- `pool_points`: uses per-player projected GP (injury-risk-adjusted)
- `pool_points_full_season`: uses flat `games_per_season` = 84 for all players

Rankings in the output files are sorted by `pool_points` by default.

9.3 Forward scoring

$$\text{pool\_points}_i^{\text{fwd}} = \hat{y}_i^{\text{pts/g}} \times \text{GP}_i^{\text{proj}} \tag{15}$$

9.4 Defense scoring

$$\widehat{\text{pts}}_i = \hat{y}_i^{\text{pts/g}} \times \text{GP}_i^{\text{proj}} \tag{16}$$

$$\widehat{\text{pm}}_i = \hat{y}_i^{\text{pm/g}} \times \text{GP}_i^{\text{proj}} \tag{17}$$

$$\text{pool\_points}_i^{\text{def}} = \widehat{\text{pts}}_i + \widehat{\text{pm}}_i \tag{18}$$

Blended predictions are used for $\hat{y}^{\text{pts/g}}$ and $\hat{y}^{\text{pm/g}}$.

9.5 Goalie scoring

Let $W_i = \hat{y}_i^{\text{wins/g}} \times \text{GP}_i^{\text{proj}}$ and $S_i = \min(\hat{y}_i^{\text{so/g}} \times \text{GP}_i^{\text{proj}},\ W_i)$ (shutouts capped at wins):

$$\text{pool\_points}_i^{\text{goalie}} = 1{\cdot}(W_i - S_i) + 3{\cdot}S_i + 10{\cdot}\mathbf{1}[\text{best GAA among qualified}] + 10{\cdot}\mathbf{1}[\text{best Sv\% among qualified}] \tag{19}$$

A goalie is *qualified* for the bonus if $\text{GP}_i^{\text{proj}} \geq 40$. The GAA bonus goes to $\arg\min_i \hat{y}_i^{\text{GAA}}$ among qualified goalies; the save% bonus to $\arg\max_i \hat{y}_i^{\text{sv\%}}$.

10 Diagnostics

Model diagnostics are generated automatically at the end of every `--stage predict` run.

10.1 Outputs

`results/diagnostics/model_metrics.csv`
: One row per model. Columns: model, label, algorithm, n_train, n_test, test_r2, train_r2, baseline_r2, beats_baseline, overfitting_ratio, mae, rmse, cv_mean, spearman_r, concordance, directional_acc, bias, oos_r2, oos_mae, oos_n, oos_chosen_model, oos_spearman_r, oos_concordance, oos_directional_acc, oos_bias, oos_top1_match_max, oos_top1_match_min.

`plots/diagnostics/pva_<target>.png`
: Two-panel plot per model: (left) predicted vs. actual scatter with $y = x$ reference line and R^2 annotation; (right) residuals vs. predicted (checks heteroskedasticity). Generated from `y_test`/`y_pred_test` stored in the joblib artifact — no retraining required.

`plots/diagnostics/rolling_oos_all.png`
: Grid of OOS R^2 by holdout year for each model (one subplot per model). Shows whether model quality has been stable over time or has degraded in recent seasons.

`results/diagnostics/gp_projection_check.csv`
: All players sorted by $\text{GP}^{\text{proj}}/\text{GP}^{\text{last}}$. Players below 0.70 are flagged in the log as anomalies requiring manual review before draft day. Produced by `pool_ranking._write_gp_diagnostic()`.

`results/diagnostics/source_reconciliation.csv`
: NHL API vs. MoneyPuck agreement on counting stats. Produced by `common/diagnostics/reconcile.py`.

`plots/feature_importance_<target>.png`
: Horizontal bar chart of the top 15 features by importance (tree models: built-in feature importances; linear models: |coef|).

10.2 Console summary

A compact table is printed to stdout at the end of every predict run:

```
=== Model fit summary ===
label             algorithm  test_r2  baseline_r2  beats_baseline
   spearman_r  concordance  directional_acc  oos_r2  oos_spearman_r
   oos_concordance
Fwd Points/G      gradient_boosting   0.712    0.001     True  0.843
   0.912   0.81   0.665   0.812   0.897
Def Points/G      ridge               0.531    0.001     True  0.741
   0.869   0.74   0.402   0.711   0.851
...
```

11 Results

11.1 Model performance

Table 3 summarises out-of-sample performance for all seven production models as of the first full diagnostic run (2026-07-24). Concordance and Spearman ρ for the OOS evaluation were not yet computed for all targets at that run; estimated ranges are given where exact values are unavailable.

Table 3: OOS model performance — all 7 targets (2026-07-24)

Target	Algorithm	OOS R^2	OOS Conc.	OOS Spear. ρ	Notes
Fwd Points/G	Gradient Boosting	0.665	≈ 0.90	≈ 0.83	Healthy
Def Points/G	Ridge / GBM	≈ 0.40	≈ 0.72	≈ 0.71	Post-blend
Def +/−/G	ElasticNet	0.031	≈ 0.54	≈ 0.10	Near-chance; hist dominates
Goalie Wins/G	Ridge	≈ 0.10	≈ 0.60	≈ 0.35	Team-driven
Goalie SO/G	Ridge	≈ 0.05	≈ 0.55	≈ 0.20	Rare event
Goalie GAA	Ridge	-0.203	≈ 0.52	≈ 0.10	Below mean; expected
Goalie Sv%	Lasso	-0.839	≈ 0.51	≈ 0.05	Below mean; EB helps

Concordance and Spearman ρ values marked $\approx$ are estimates; exact values in model_metrics.csv.

11.2 Top-10 overall ranking (2026-07-24)

Overall Rank	Player	Position
1	N. MacKinnon	F
2	N. Kucherov	F
3	C. McDavid	F
4	C. Makar	D
5	L. Draisaitl	F
6	D. Pastrnak	F
7	E. Bouchard	D
8	N. Suzuki	F
9	M. Necas	F
10	M. Celebrini	F

Cale Makar at #4 reflects the defense blending fix; prior to blending he ranked #44.

12 Known Limitations

Goalie precision stats

Season-to-season autocorrelation for GAA and save% is $r \approx 0.09$ — this is a fundamental property of goalie performance data, not a model failure. Published ranges (Thomas 2006; Macdonald 2010; Schuckers & Curro 2013) are $r \approx 0.05$–0.15 for raw Sv% and $r \approx 0.15$–0.30 for GSAx. No model can reliably predict who will have the best GAA next season from prior stats alone. The pool bonus only requires correctly identifying the argmax/argmin among a small number of qualified goalies, which is a weaker requirement than accurate regression.

Defense plus/minus

OOS $R^2 = 0.031$ and concordance ≈ 0.54. The model barely beats the chance baseline. This stat is highly dependent on team context (a plus/minus changes by ± 1 whether or not the goalie saves a shot that would not be attributed to any on-ice skater). Prediction blending (alpha = 0.10 toward historical averages) partially corrects the ranking.

Traded players — historical team context

Players who changed teams during a historical season are assigned the *first* team listed in the NHL API's comma-joined `team_abbrev` field. Season totals (goals, assists, games_played) are correctly preserved. Only the team-context lag features (used by the defense model) may be inaccurate for multi-team seasons. Per-team game logs are only available for the most recent season, so a majority-team fix across all 18 training seasons would require expanding game-log history.

MoneyPuck coverage

$\sim$5.5% of players are unmatched in MoneyPuck. These are fringe players with very few games played who are genuinely absent from MoneyPuck's records, not join failures. They receive NaN for all MoneyPuck features, which become 0 after imputation.

Observation weighting not implemented

All training rows carry equal weight regardless of the player's game count. A backup goalie with 3 starts influences the loss function equally to a starter with 60. Planned fix: `sample_weight = min(GP, 40) / 40` via optional threading in `training.py` (see Future Work).

Pool scoring rules constant year-to-year

No time-varying config is needed; the rules have not changed. Any future rule change requires only a config edit.

13 Future Work

GP diagnostic Layer 2: game-log debut date

The current partial-season filter (GP < 25) works on the aggregate season count. A richer signal is already available: `nhl_game_logs.csv` contains `gameDate` in ISO format; the existing `get_player_game_log()` endpoint in `nhl_api.py` can fetch any player/season combination. For any player whose debut season has GP < 50, fetching their game log and checking the first game date would definitively distinguish a March call-up (first game after February) from an October injury (first game in November). Deferred because the threshold filter handles the known cases; implement if edge cases emerge.

Observation weighting

Thread `sample_weight` through `_fit_one()`, `train_and_select()`, and `train_all_vs_exclud` in `training.py` as an optional parameter (default None = current behavior). In goalie `build_xy_for()`, compute weights as `min(GP, 40) / 40` and return as a 4th element. This down-weights 1–5 start goalies who contribute mostly noise to the fit.

MultiTaskElasticNet for (GAA, save%)

Train a single `sklearn.linear_model.MultiTaskElasticNetCV` on the $[GAA, \text{save\%}]$ pair to enforce joint feature sparsity: a feature is either selected for both targets or neither. The algebraic linkage ($r = -0.84$) means both targets should share most of their predictive signal. Only implement if observation weighting alone is insufficient.

Joint D-men model

Defenseman goals and assists directly contribute to their plus/minus (a scored goal counted in points is also counted as +1). Predicting plus_minus as a residual after removing the points contribution would isolate the defensive-zone and special-teams components. Alternatively, a multi-output regression joint over (points, plus_minus) could be explored.

Majority-team for historical traded players

Expand per-game log collection from the current season only to all historical seasons (one API call per player per season for $\sim$18 years). Assign each player to the team they played the most games for. This would make the team-context lag features more accurate for the $\sim$5–10% of historical rows involving mid-season trades.

Interactive draft-day tool

A live tool for draft day: show remaining available players ranked by pool points, allow filtering by position, mark picked players. Lowest priority; nothing built yet.

2027-28 season

Copy `2026-27/config.yaml` $\rightarrow$ `2027-28/config.yaml`, update `season`, `season_id`, `roster_as_of_date`, `games_per_season` (if the NHL expands again), and any `trade_overrides`. Run `python3 run_season.py --season 2027-28 --stage all`.

References

- Thomas, A. C. (2006). "The impact of puck possession and location on ice hockey strategy." *Proceedings of the Joint Statistical Meetings (Statistics in Sports)*. [Empirical Bayes save% shrinkage; near-zero YoY autocorrelation for Sv%.]
- Macdonald, B. (2010). "A regression-based adjusted plus-minus statistic for NHL players." *Journal of Quantitative Analysis in Sports*, 7(3).
- Schuckers, M. & Curro, J. (2013). "Total Hockey Rating (THoR): A comprehensive statistical rating of National Hockey League forwards and defensemen based upon all on-ice events." *MIT Sloan Sports Analytics Conference*.
- Gramacy, R. B., Jensen, S. T., & Taddy, M. (2013). "Estimating player contribution in hockey with regularized logistic regression." *Journal of Quantitative Analysis in Sports*, 9(1).
- MoneyPuck.com methodology notes: `https://moneypuck.com/about.htm`. [GSAx definition; shot danger zone classification.]